

Step 1: Check GCC

```
gcc --version
```

```
g++ --version
```

If not installed:

```
sudo apt update
```

```
sudo apt install build-essential
```

Step 2: Save your code

```
nano bfs_dfs.cpp
```

```
CTRL + O → Enter → CTRL + X
```

Step 3: Compile with OpenMP

```
g++ -fopenmp bfs_dfs.cpp -o hpc
```

Step 4: Run

```
./hpc
```

1. CAN YOU RUN THIS IN LAB?

Ask yourself:

✓ **Run this command:**

```
nvidia-smi
```

👉 **If you see GPU info:**

✓ You can run CUDA

👉 **If you see:**

command not found

✗ No GPU → **your code will NOT run**

👉 In most colleges → CUDA is **NOT available**

✅ 2. HOW TO RUN (ONLY IF CUDA EXISTS)

◆ Step 1: Save file

`nano vector_add.cu`

Paste code → save

◆ Step 2: Compile

`nvcc vector_add.cu -o run`

◆ Step 3: Run

`./run`

🧠 3. WHAT THIS PROGRAM DOES (VERY SIMPLE)

👉 You have 2 arrays:

`A = [1, 2, 3, 4]`

`B = [5, 6, 7, 8]`

👉 Output:

`C = [6, 8, 10, 12]`

👉 This is **Vector Addition**

⚡ CPU vs GPU

Your code runs SAME work in 2 ways:

◆ CPU (normal)

for (i=0 to N)

`C[i] = A[i] + B[i]`

◆ GPU (parallel)

Each thread computes one element

👉 Thousands of threads → faster

📊 4. EXPECTED OUTPUT (IMPORTANT)

When you run, you will see something like:

CPU Time: 120.50 ms

GPU Time: 5.20 ms

Speedup: 23.17x

Verification: PASS

MEANING OF OUTPUT

✓ CPU Time

Time taken by normal code

✓ GPU Time

Time taken by CUDA

✓ Speedup

CPU / GPU

👉 Bigger = better

✓ Verification

PASS → correct output

FAIL → wrong

5. MATRIX MULTIPLICATION OUTPUT

Second program gives:

Matrix Multiplication (512x512)

CPU time: 2500 ms

GPU Naive time: 120 ms Speedup: 20x

GPU Tiled time: 60 ms Speedup: 40x

Naive Verify: PASS

Tiled Verify: PASS

WHAT IS HAPPENING?

👉 Multiply matrices:

$$C[i][j] = \text{sum}(A[i][k] * B[k][j])$$

6. WHAT TO SAY IN EXAM (VERY IMPORTANT)

If it doesn't run:

👉 Say confidently:

"This program requires NVIDIA GPU and CUDA runtime. If GPU is not available, it cannot execute, but logic remains same."

🧠 7. SIMPLE SUMMARY

👉 Vector Add:

- Adds two arrays
- GPU faster

👉 Matrix Multiply:

- Multiply matrices
- Tiled version fastest

code :

1. SIMPLE VECTOR ADD

```
#include <stdio.h>
```

```
#include <cuda_runtime.h>
```

```
#define N 10 // small size for understanding
```

```
// GPU Kernel
```

```
__global__ void add(int *A, int *B, int *C) {
```

```
    int i = threadIdx.x;
```

```
    C[i] = A[i] + B[i];
```

```
}
```

```
int main() {
```

```
    int A[N] = {1,2,3,4,5,6,7,8,9,10};
```

```
    int B[N] = {10,9,8,7,6,5,4,3,2,1};
```

```

int C[N];

int *d_A, *d_B, *d_C;

cudaMalloc(&d_A, N*sizeof(int));
cudaMalloc(&d_B, N*sizeof(int));
cudaMalloc(&d_C, N*sizeof(int));

cudaMemcpy(d_A, A, N*sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_B, B, N*sizeof(int), cudaMemcpyHostToDevice);

// Launch kernel (1 block, N threads)
add<<<1, N>>>(d_A, d_B, d_C);

cudaMemcpy(C, d_C, N*sizeof(int), cudaMemcpyDeviceToHost);

printf("Result:\n");
for(int i=0;i<N;i++)
    printf("%d ", C[i]);

cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);

return 0;
}

```

2. SIMPLE MATRIX MULTIPLICATION

```

#include <stdio.h>

#include <cuda_runtime.h>

```

```
#define N 2 // small for understanding
```

```
__global__ void matMul(int A[N][N], int B[N][N], int C[N][N]) {
```

```
    int row = threadIdx.y;
```

```
    int col = threadIdx.x;
```

```
    int sum = 0;
```

```
    for(int k=0;k<N;k++)
```

```
        sum += A[row][k] * B[k][col];
```

```
    C[row][col] = sum;
```

```
}
```

```
int main() {
```

```
    int A[N][N] = {{1,2},{3,4}};
```

```
    int B[N][N] = {{5,6},{7,8}};
```

```
    int C[N][N];
```

```
    int (*d_A)[N], (*d_B)[N], (*d_C)[N];
```

```
    cudaMalloc(&d_A, sizeof(A));
```

```
    cudaMalloc(&d_B, sizeof(B));
```

```
    cudaMalloc(&d_C, sizeof(C));
```

```
    cudaMemcpy(d_A, A, sizeof(A), cudaMemcpyHostToDevice);
```

```
    cudaMemcpy(d_B, B, sizeof(B), cudaMemcpyHostToDevice);
```

```
    dim3 threads(N, N);
```

```

matMul<<<1, threads>>>(d_A, d_B, d_C);

cudaMemcpy(C, d_C, sizeof(C), cudaMemcpyDeviceToHost);

printf("Result Matrix:\n");

for(int i=0;i<N;i++){
    for(int j=0;j<N;j++){
        printf("%d ", C[i][j]);
        printf("\n");
    }

    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);

    return 0;
}

```

HOW TO RUN

```

nvcc file.cu -o run
./run

```

◆ Step 1: Install WSL

Open PowerShell (Admin):

```
wsl --install
```

Restart PC

◆ Step 2: Open Ubuntu terminal

Search → "Ubuntu"

◆ Step 3: Install GCC + OpenMP

sudo apt update

sudo apt install build-essential

◆ Step 4: Run your HPC code

nano program.cpp

g++ -fopenmp program.cpp -o run

./run